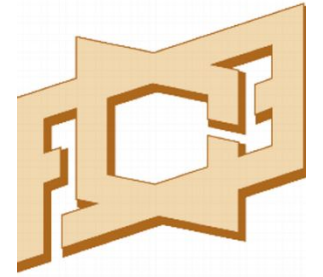




UNIVERSIDAD DEL ZULIA
FACULTAD EXPERIMENTAL DE CIENCIAS
DIVISIÓN DE PROGRAMAS ESPECIALES
LICENCIATURA EN COMPUTACIÓN



DESARROLLO DE SISTEMAS

PROYECTO FINAL

Sistema de Gestión de Entrenador Personal

PERIODO I -
2026

Presentado por

- Montiel Cordero, Jorge Luis
- C.I.: 31.545.512

AGENDA

1. Ciclo de Vida seleccionado (resumen)
2. Tecnologías y Herramientas de Desarrollo
3. Diagrama de Flujo de Datos (DFD)
4. Diccionario de Datos (DFD)
5. Diccionario de Datos (BD)
6. Diagrama Entidad – Relación
7. Diccionario de Datos (Diagrama E-R)
8. Interfaces de Entrada / Salida
9. Demostración de Pruebas al SI
10. Plan de Mantenimiento
11. Personal que participo (soporte - minuta)

ENTRADAS

- Datos del instruido: Nombre, edad, peso, nivel de actividad física, sexo, altura
- Propósito del entrenamiento
- Datos Nutricionales
- Registro de anteriores entrenamientos
- Días disponibles a la semana
- Historial médico
- Configuración de rutinas

PROCESOS

- Cálculo metabólico para ayudar al entrenador con los gastos calóricos total según la actividad física del cliente.
- Análisis del propósito del entrenamiento según el registro de entrenamiento o del nivel de actividad física y su historial de lesiones

SALIDAS

- Rutina semanal o mensual predeterminada según los propósitos personales (pero sujeta a personalización)
- Gráfica del rendimiento: para manejar el rendimiento mensual del cliente
- Dieta semanal y resumen nutricional

Incremental + Scrum Híbrido

Utilizaremos un ciclo de vida incremental para realizar una entrega rápida de un producto mínimo viable (MVP), para que el usuario pueda visualizar y utilizar las partes del sistema que se van creando.

Incrementos:



Al dividir el proyecto en 5 incrementos específicos, el riesgo de fallos críticos se distribuye. Es mucho más sencillo incorporar cambios basados en el feedback real.

Incremental + Scrum Híbrido

Al inicio de cada incremento se realiza un Sprint Planning, Daily Scrum para hacer reuniones para decidir cómo se va a plantear el desarrollo, Sprint Review para al final de cada incremento realizar una prueba completa de cada incremento y cómo se acopla al sistema, Retrospectiva sería el feedback donde se realizan los cambios por si no le gustan al cliente



Planificación

Al inicio de cada incremento se define el alcance técnico y los objetivos de negocio del módulo.



Ejecución Daily

Reuniones diarias para coordinar el planteamiento del desarrollo y resolver bloqueos técnicos.



Review & Retro

Pruebas de acople al sistema y ajustes basados en el feedback directo del cliente al finalizar el bloque.

PROYECTO FINAL

TECNOLOGÍAS Y HERRAMIENTAS DE DESARROLLO

Backend

- Python
- Flask
- JavaScript
- Nodejs



Frontend

- HTML
- CSS
- JavaScript
- React



Base de datos

- MySQL
- Redis



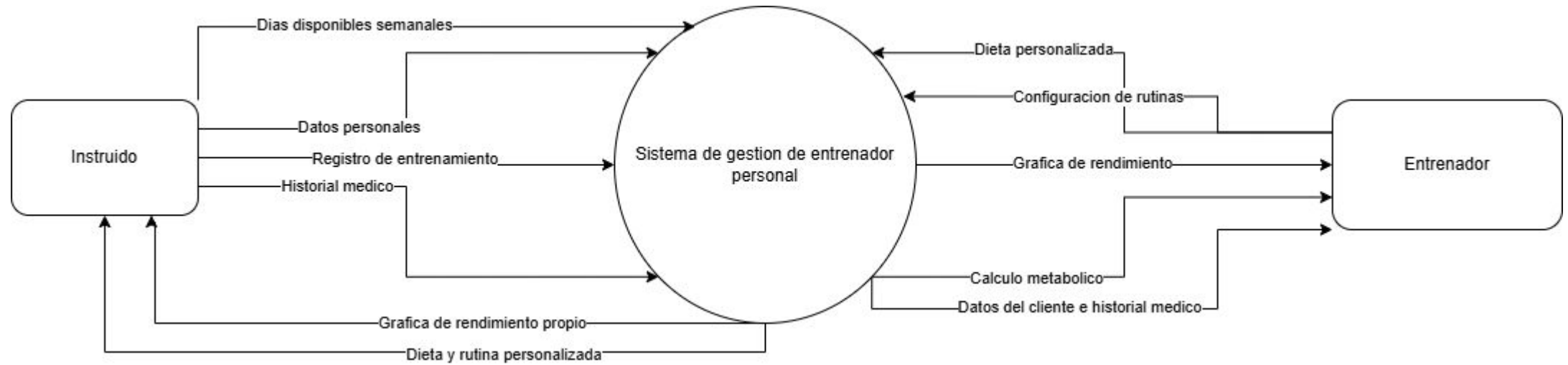
Herramientas

- Visual Studio Code
- GitHub
- DBeaver
- Aiven
- Vercel
- Raiwa
- Opencode



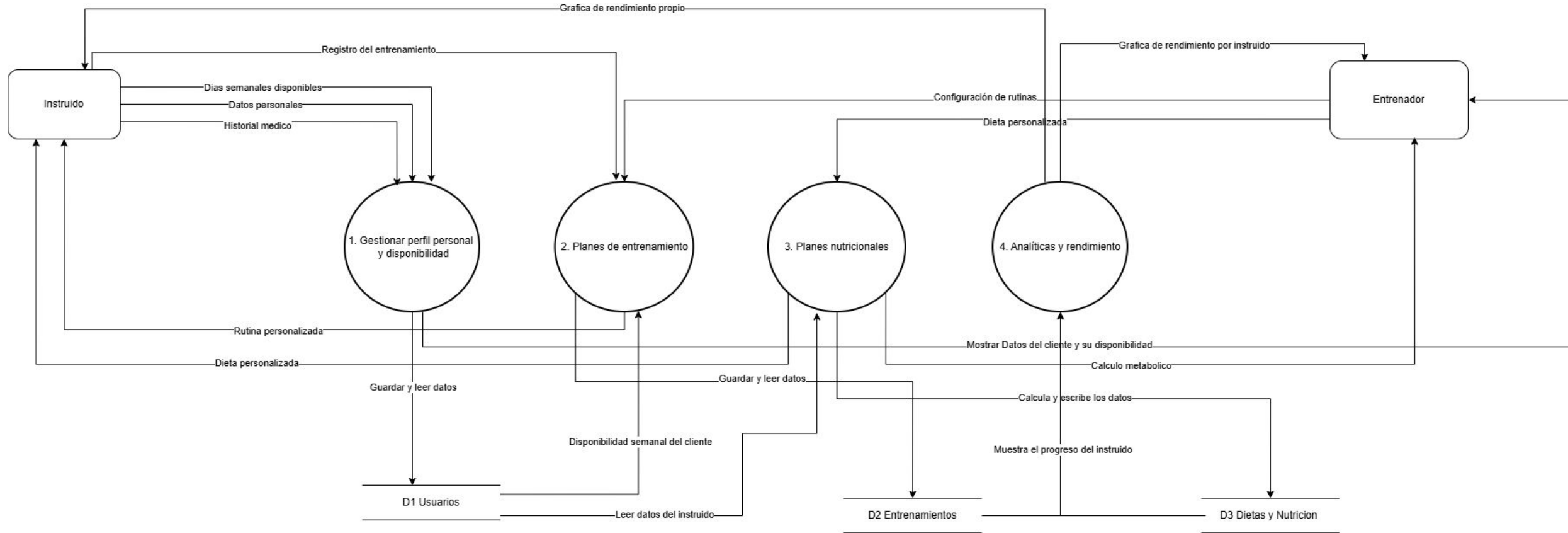
PROYECTO FINAL

DIAGRAMA DE FLUJO DE DATOS – DFD NIVEL 0



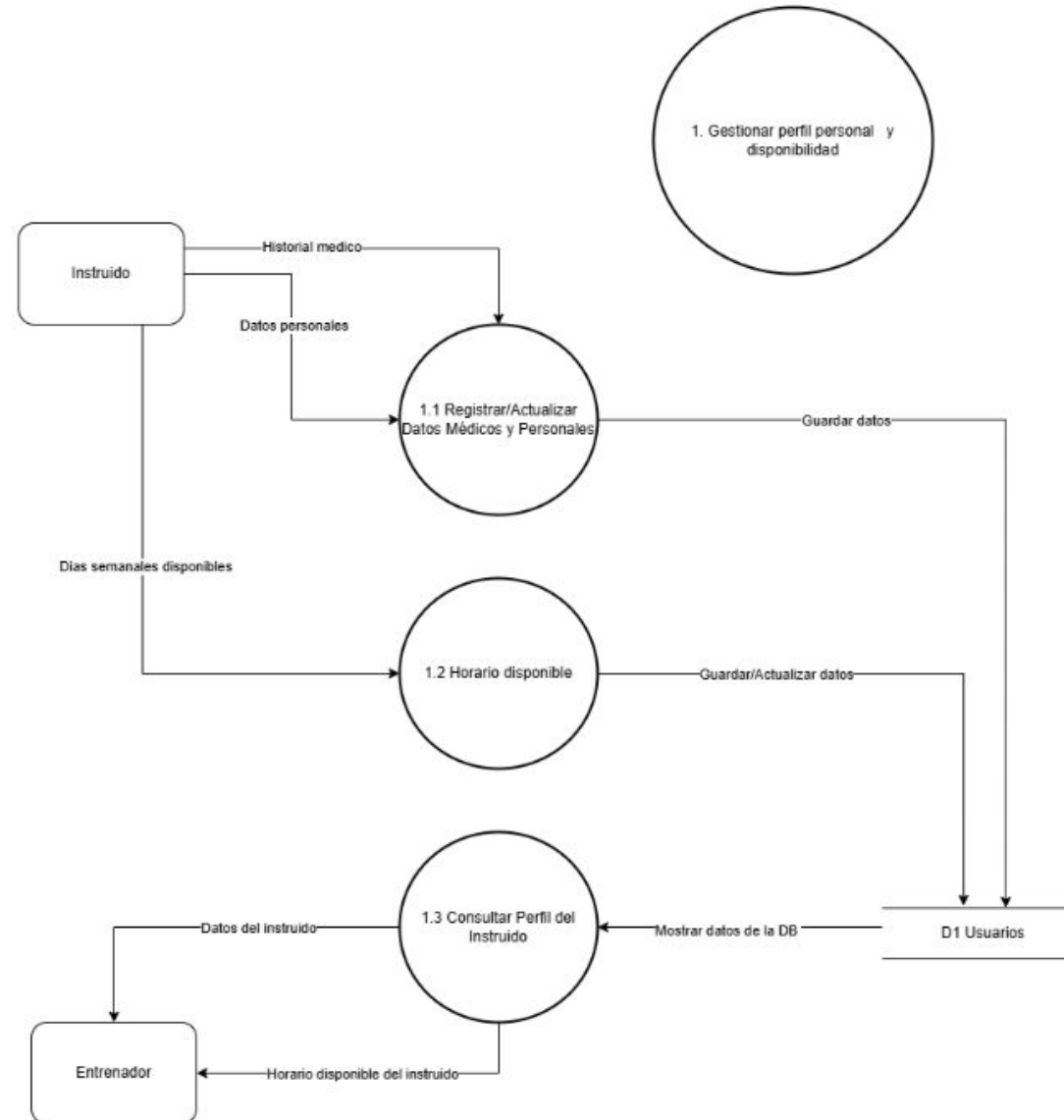
PROYECTO FINAL

DIAGRAMA DE FLUJO DE DATOS – DFD NIVEL 1



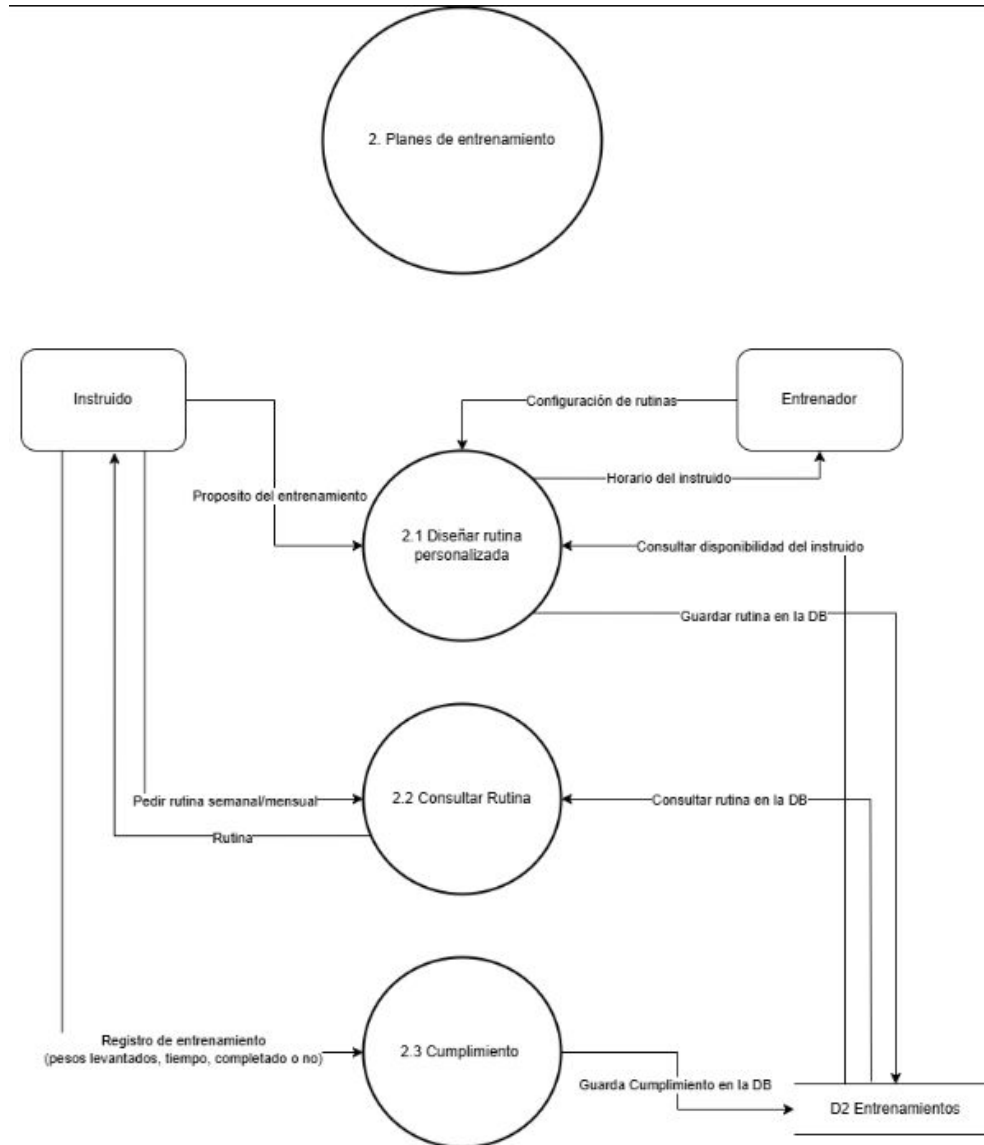
PROYECTO FINAL

DIAGRAMA DE FLUJO DE DATOS – DFD NIVEL 2 Gestionar perfil personal y disponibilidad



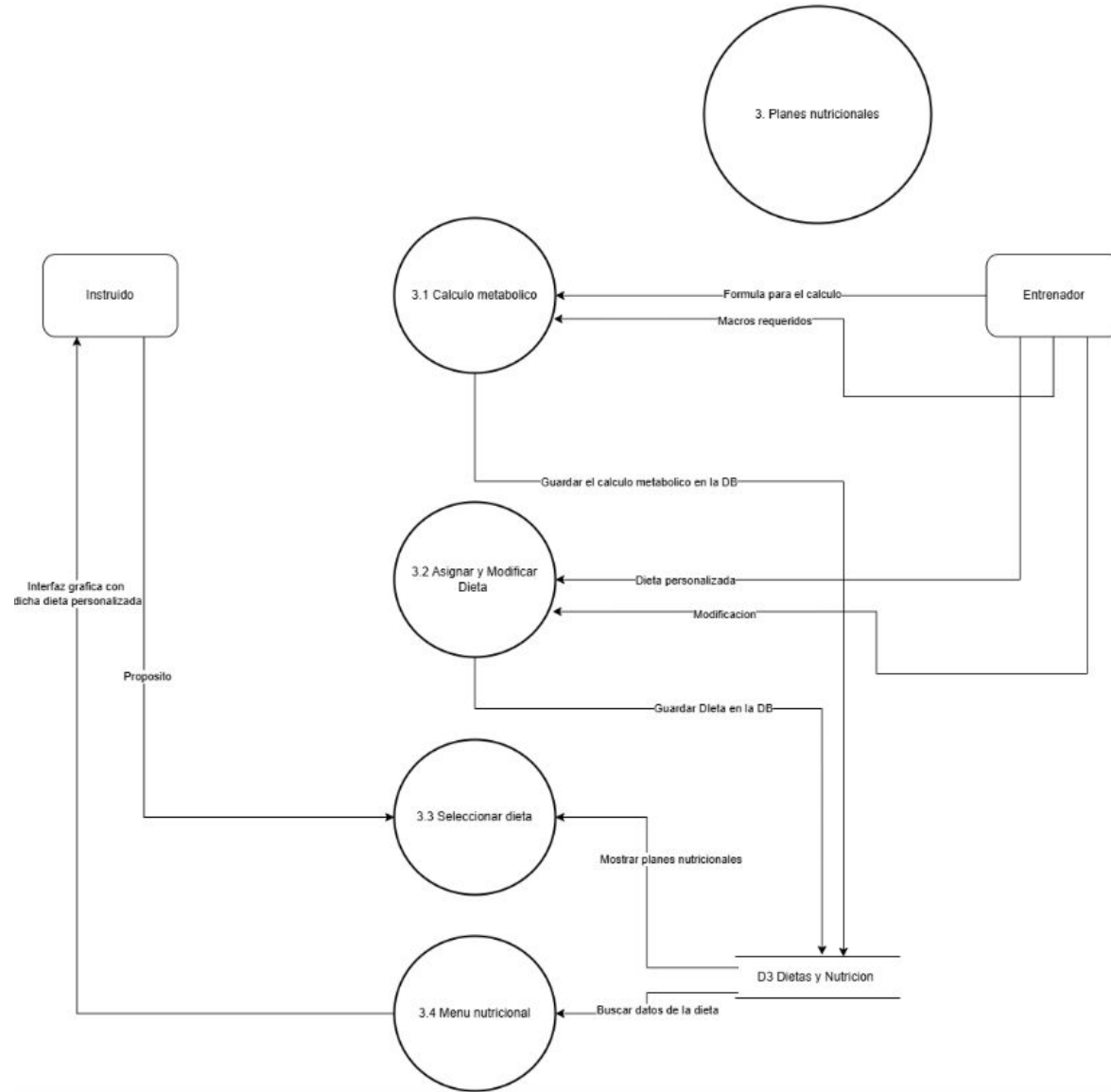
PROYECTO FINAL

DIAGRAMA DE FLUJO DE DATOS – DFD NIVEL 2 Planes de entrenamiento



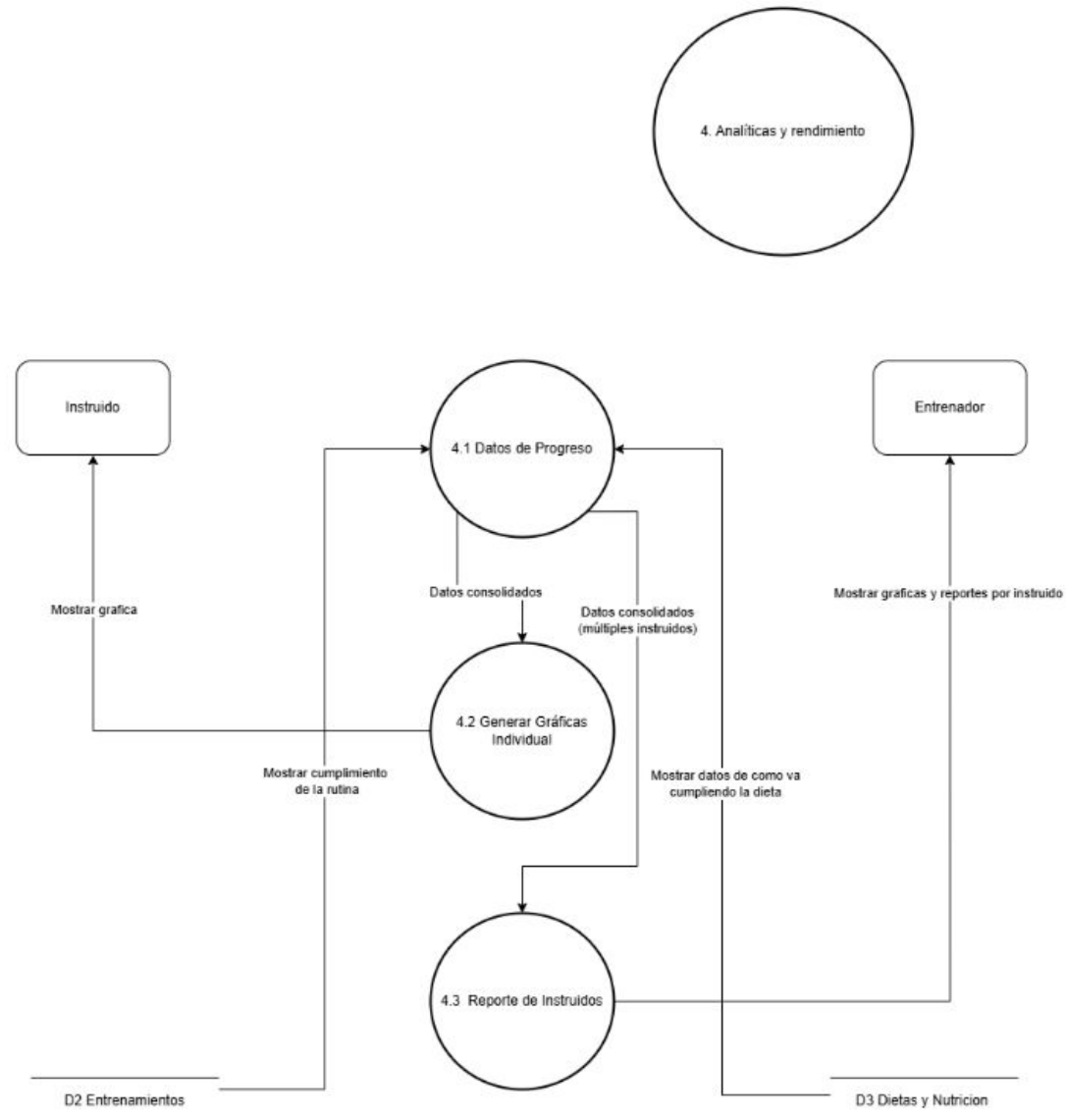
PROYECTO FINAL

DIAGRAMA DE FLUJO DE DATOS – DFD NIVEL 2 Planes nutricionales



PROYECTO FINAL

DIAGRAMA DE FLUJO DE DATOS – DFD NIVEL 2 Analíticas y rendimiento



PROYECTO FINAL

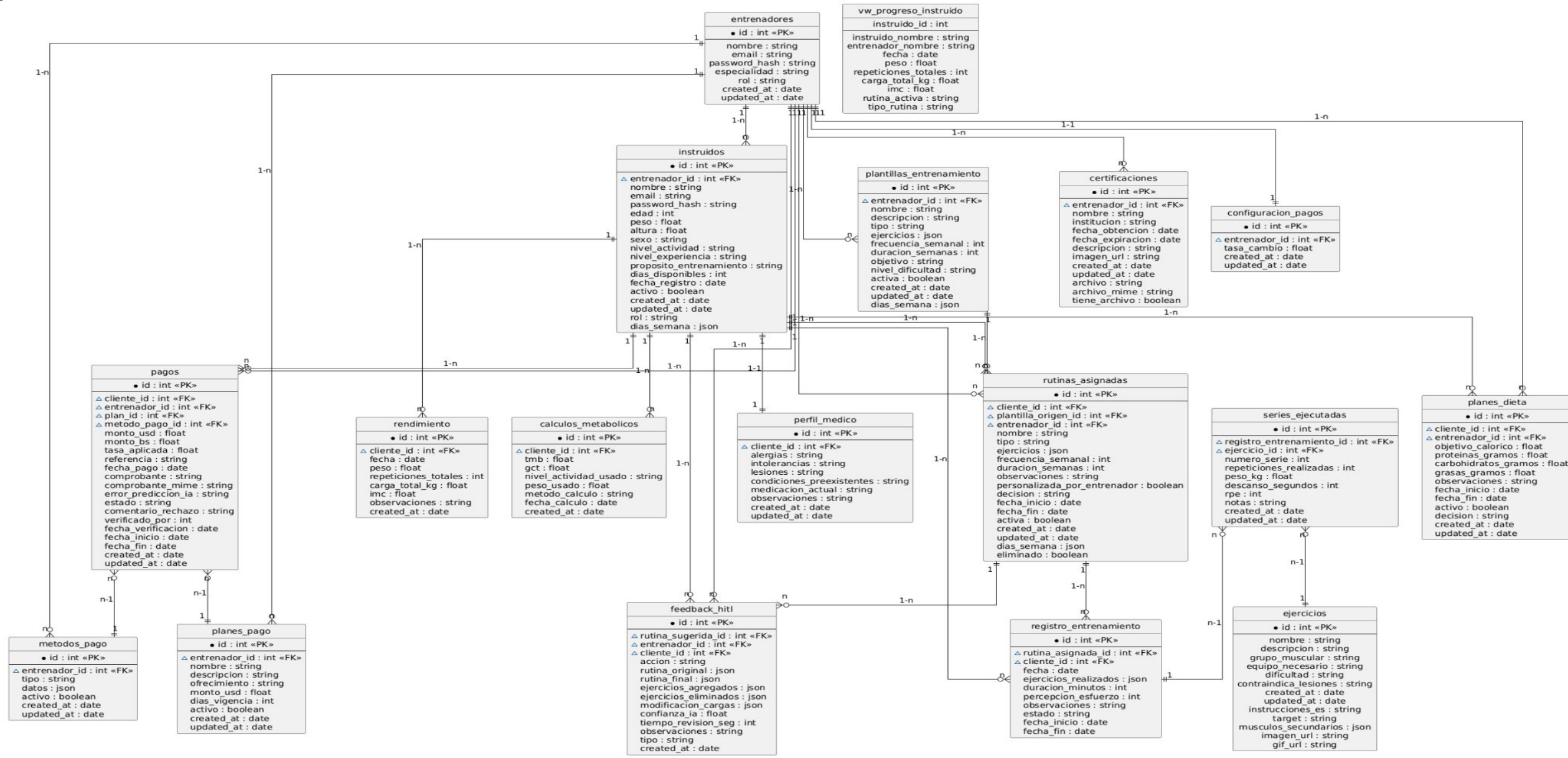
DICCIONARIO DE DATOS - DFD

Ficha de Entidad:

Instruido		
Descripción: Usuario principal del sistema que recibe los planes de entrenamiento		
Entrada Propósito del entrenamiento, pedir rutina semanal/mensual, registro de entrenamiento	Descripción de la entidad El instruido que define su propósito, solicita visualizar su rutina y registra el cumplimiento de sus ejercicios	Salida Rutina

PROYECTO FINAL

DIAGRAMA ENTIDAD – RELACIÓN



PROYECTO FINAL

INTERFAZ DE ENTRADA



Crear Cuenta

Regístrate para empezar a usar el sistema

Nombre completo

Juan Pérez

Email

tu@email.com

Contraseña

Mínimo 8 caracteres

Confirmar contraseña

Repite tu contraseña

Edad

25

Peso (kg)

70.5

Altura (m)

1.75

Sexo

Seleccionar...

Nivel de actividad

Seleccionar...

Propósito de entrenamiento

Seleccionar...

Nivel de experiencia

Seleccionar...

Días disponibles para entrenar

Lun

Mar

Mie

Jue

Vie

Sab

Dom

Has seleccionado 0 días

Crear Cuenta →

¿Ya tienes cuenta? [Inicia sesión](#)

PROYECTO FINAL

INTERFAZ DE SALIDA

YanTraining

Dashboard

Entrenador

Entrenamiento

Dietas

Reportes

Mi Plan

Mi Perfil

Dashboard

EN LÍNEA

PESO

Última medición

90.30 kg

Índice de masa corporal

27.9

ALTURA

Última medición

1.80 m

IMC 27.9 · Sobrepeso

CONSISTENCIA

Ritmo de entrenamiento

3 sesiones

Meta mensual

16 sesiones

Historial de Entrenamientos

Registros recientes de carga y esfuerzo

Ver todo →

FECHA	RUTINA	DURACIÓN	ESFUERZO (RPE)	OBSERVACIONES
14 sept	● sda	1 min	—	Resumen: {"volumenTotal":2970,"porEjercicio":{"*119":{"series":3,"repeticiones":24,"volumen":1770},"*361":{"series":2,"repeticiones":15,"volumen":1200}}}
04 sept	● Prueba	1 min	—	Resumen: {"volumenTotal":1680,"porEjercicio":{"*119":{"series":2,"repeticiones":24,"volumen":1560},"*293":{"series":1,"repeticiones":12,"volumen":120}}}
03 sept	● Prueba	—	—	—

Mi Progreso

Evolución de tus sesiones recientes

Esfuerzo

Duración

Carga

Jorge Luis

prueba@gmail.com

PRUEBAS DE SISTEMAS DE INFORMACIÓN

1. Pruebas del Backend Principal (Node.js)

El backend en Node.js concentra la mayor parte de la cobertura de pruebas en el directorio backend-node/tests/. Las pruebas están fuertemente enfocadas en validar la lógica de negocio, la seguridad perimetral y el rendimiento mediante el manejo de caché.

- **Autenticación y Seguridad:**

- Se verifica la lógica central del servicio de autenticación (auth.service.test.js).
- Se prueban las reglas de validación de entrada (auth.validation.test.js).
- Se evalúa la persistencia y expiración en la caché de autenticación (auth.cache.test.js).
- Se comprueba la correcta invalidación de tokens en caché (auth.invalidacion.test.js).
- Existe validación específica para el bloqueo de accesos mediante listas negras (blacklist.test.js).

- **Gestión de Usuarios (Instruidos y Perfiles Médicos):**

- Se prueban los permisos y autorizaciones vinculados a los instruidos (instruido.service.auth.test.js).
- Se verifica la integridad de los datos de los usuarios instruidos (instruido.validation.test.js).
- Se comprueban las funcionalidades de los perfiles médicos (perfil-medico.service.test.js).

- **Gestión de Entrenamientos y Dietas:**

- Se valida la asignación de rutinas (rutinas-asignadas.service.test.js) y las autorizaciones requeridas en sus controladores (rutinas-asignadas.controller.auth.test.js).
- Se aprueba el correcto registro del progreso de entrenamiento (registro-entrenamiento.test.js).
- Se verifica la gestión de ejercicios (ejercicios.service.test.js) y la duración de estos datos en caché (ejercicios.ttl.test.js).
- Se validan los servicios relacionados con los planes de alimentación (dietas.service.test.js).
- El manejo de plantillas cuenta con pruebas para los controladores, almacenamiento en caché y su respectiva invalidación (plantillas.controller.auth.test.js, plantillas.cache.test.js, plantillas.invalidacion.test.js).

PROYECTO FINAL

PRUEBAS DE SISTEMAS DE INFORMACIÓN

- **Módulo de Pagos:**
 - Se evalúa la persistencia temporal en caché de la información de transacciones (pagos.cache.test.js).
 - Se verifica la correcta destrucción de la caché tras la actualización de un estado de pago (pagos.invalidacion.test.js).
- **Reportes Analíticos:**
 - Se prueban las funciones generadoras de reportes (reportes.test.js).
 - Se certifica el rendimiento de las métricas probando su almacenamiento en caché (reportes.cache.test.js) y su ciclo de vida (reportes.ttl.test.js).
- **Integración e Infraestructura:**
 - Se valida el funcionamiento global de la caché y el registro de sus métricas (cache.test.js, cache.metricas.test.js).
 - Se prueba el cliente HTTP interno responsable de comunicar el entorno Node con el motor de Flask (flask-client.test.js).
 - Se evalúan las validaciones enviadas a la capa HITL (*Human-In-The-Loop*) (hitl.service.validate.test.js).

2. Pruebas del Backend Secundario (Python/Flask)

Las pruebas relacionadas con el motor de IA, reglas y cálculos avanzados están localizadas en backend-flask/tests/:

- **Servicios Críticos:** Se cuenta con un conjunto de pruebas diseñadas para el servicio "Guardian" (test_guardian.py), el cual funciona como barrera de seguridad o validador de las recomendaciones de entrenamiento y nutrición.

3. Pruebas Criptográficas Globales

- A nivel global, en el directorio tests/, existe un módulo enfocado en certificar los algoritmos de utilidad y seguridad criptográfica (crypto.test.js).

4. Automatización y Agentes de Testing

El repositorio está configurado para la automatización e integración continua de estas pruebas, delegando responsabilidades en herramientas internas de OpenCode:

- Se define un agente autónomo exclusivo para las labores de aseguramiento de calidad (QA) y pruebas (.opencode/agent/tester.md).
- Se incluye una configuración de comandos dedicada para disparar las baterías de prueba (.opencode/command/test.md).

PROYECTO FINAL

PLAN DE MANTENIMIENTO

Necesidades a evaluar del mantenimiento del sistema web:

1. Paso 1: Auditar el rendimiento actual del sitio web

Empezaría a evaluar la velocidad, la compatibilidad móvil y la funcionalidad general del sistema web. Pienso que puedo utilizar herramientas como Google PageSpeed Insights para comprobar los tiempos de carga en dispositivos de sobremesa y móviles. Además, asegurandome que el sistema web es apto para móviles probándolo en distintos dispositivos y evaluándolo en el informe para móviles de Google Search Console.

2. Paso 2: Identificar las necesidades de seguridad del sistema y el cumplimiento normativo

Tendré que realizar un manual de seguridad del sistema para llevar un seguimiento de cosas a revisar y cumplir con la seguridad del sistema, por ejemplo como certificados SSL, reglas de cortafuegos y/o escáneres de seguridad. Luego revisar si el sistema necesita cumplir un normativo, por último se evalúan las necesidades de privacidad de datos y copias de seguridad en función de tu postura de seguridad requerida.

3. Paso 3: Evaluar la gestión de como voy a evaluar el contenido

Definir si el contenido del sistema debe ser un mantenimiento semanal, mensual o trimestral. Eso se define luego de evaluar las páginas o áreas clave en las que necesite garantizar el rendimiento y desarrollar un plan de actualizaciones o añadir algunas funciones nuevas.

4. Paso 4: Analizar las necesidades de mantenimiento técnico

Evaluar el código base de tu sitio web en busca de algunas bibliotecas obsoletas o algunas bibliotecas que funcionen mejor. Comprobar regularmente los registros de errores del sistema web para identificar problemas ocultos que puedan afectar al rendimiento o a la compatibilidad. Además, se evalúa si el entorno de alojamiento satisface las necesidades de tu sitio web en términos de ancho de banda, almacenamiento y tiempo de actividad del servidor.

5. Paso 5: Estimar recursos y presupuesto

Después de crear una lista de tareas de mantenimiento del sistema web, el siguiente paso es estimar el coste y los recursos. Identificar el coste de cualquier herramienta o proveedor externo que sea necesario. Este es el último paso es el plan de ejecución del mantenimiento del sistema web.

PROYECTO FINAL

PLAN DE MANTENIMIENTO

El mantenimiento del sistema va a contar con varias fases para lograr conseguir un control y una guía mucho más segura, ya que el sistema en producción va a alojado el frontend en vercel, el backend en railway y la base de datos en aiven.

1. Fase 1: Preparación y notificación

- **Notificación:** Programar el mantenimiento en horas de bajo tráfico (mayormente va a ser en madrugada en horario de Venezuela/Maracaibo) configurando el aviso previo en el frontend.
- **Respaldo en aiven:** Se va a crear un Fork temporal de la base de datos MySQL en la consola de Aiven para aislar los datos reales y probar las migraciones sin afectar el clúster principal.
- **Código:** Se va a asegurar que la imagen de los contenedores estén en la rama de GitHub donde se van a asestar los cambios.

2. Fase 2: Aislamiento

- **Modo mantenimiento en vercel:** En vercel tienen un modo de mantenimiento que vamos a aprovechar, ya que se activa una redirección desde la configuración main hacia la vista OfflinePage.jsx devolviendo un HTTP 503 para bloquear la interacción de usuarios y realizar el mantenimiento seguro.
- **Pausa en railway:** Se pausa temporalmente el tráfico entrantes a los servicios web. Revisar manualmente las queries por segundo y las conexiones activas en la consola y telemetría de aiven para asegurarnos que lleguen a cero .

3. Fase 3: Ejecución de migraciones y despliegue

- **Migración de base de datos:** Conectar al host de aiven para ejecutar los scripts SQL pendientes ubicados en database/migrations/.
- **Despliegue backend (railway):** Iniciar el deploy de la imagen de docker para el backend en flask encargado del motor HITL. Una vez que esté en verde, lanzar el despliegue del backend principal en [Node.js](#).
- **Despliegue frontend (vercel):** Realizar la compilación final del proyecto en React integrando las nuevas variables de entorno si es que hay algunas variables nuevas.

4. Fase 4: Pruebas en producción (el despliegue antes de ejecutarse en producción tiene su serie de pruebas)

- Revisar manualmente el flujo de cada uno de los módulos para detectar bugs que se crearon o detectaron en producción.

5. Fase 5: Restauración y Monitoreo

- **Limpieza de Caché:** Se ejecuta una limpieza en la instancia de Redis gestionada a través de redis.js para forzar la invalidación de métricas y catálogos de ejercicios obsoletos.
- **Desactivar el modo mantenimiento:** Luego de que se pasaron las pruebas pertinentes se desactiva la redirección que ejecutamos en vercel y revisamos que funcione la configuración main y reactivamos las imágenes docker en railway para permitir que el tráfico pueda seguir efectuando los servicios webs.
- **Monitoreo activo:** Mantener abiertos los logs de Railway y el panel de consumo de RAM/CPU en Aiven durante las siguientes 24 horas para detectar cuellos de botella en el pool de conexiones o errores 500.

CONCLUSIONES

- Escalabilidad del Monolito Modular y Microservicios
- Gestión Segura y Aislamiento de Datos
- Mantenimiento Orquestado y Resiliencia
- Integración Continua y Aseguramiento de Calidad (QA)
- Sinergia del Modelo Human-in-the-Loop (HITL)